



## **POTENCIALIZANDO O DESEMPENHO COM O NOSQL**

### **Capítulo 1**

Nome do Grupo: GRW

|                                        |            |
|----------------------------------------|------------|
| Nome: Gabriel da Mata Scarpino         | RM: 567710 |
| Nome: Monalisa Santos Brasileiro       | RM: 567495 |
| Nome: Renan Assis Rocha                | RM: 567431 |
| Nome: Vinicius Eduardo Ferreira        | RM: 568117 |
| Nome: Wellington Barbosa da Silva Lima | RM: 568127 |

## **1.0 PROVA DE CONCEITO DE BANCO DE DADOS NOSQL**

### **1.1 Análise de cenário**

Ao contrário dos bancos de dados relacionais (SQL), em bancos NoSQL existem modelos de bancos de dados específicos para cada diferente situação, como o chave-valor, grafo, documento e colunar. Realizar uma escolha sem o teste poderá levar a problemas em relação a escalabilidade, lentidão de sistema e a dificuldade de manutenção, por razões como esta, fazer uma análise e testes antes da implementação é muito importante para a otimização.

#### **1.1.1 Cenário nº 1:**

Quando um cliente seleciona um produto, a plataforma de e-commerce exibe recomendações de outros itens baseadas nas compras de quem comprou esse produto e em outras promoções correlatas. No contexto atual, esse cálculo está demorando muito tempo para ser feito utilizando estruturas relacionais, dado o volume de dados envolvidos.

(Para esse primeiro cenário, a TI propôs o uso de um banco de dados NoSQL do tipo GRAFO)

#### **1.1.2 Justificativa do cenário:**

Em bancos relacionais necessitaria que o sistema realizasse múltiplos “JOINS” para conectar e sincronizar as informações entre as tabelas, o que resultaria em uma lentidão de sistema e um custo operacional cada vez mais exponencial.

Modelos de grafos permitem o cálculo em tempo real. Ele consegue identificar padrões de similaridade e promoções correlatas instantaneamente, permitindo uma personalização muito mais agressiva e atualizada.

#### **1.1.3 Empresa que utiliza o cenário:**

Como por exemplo a empresa LinkedIn que utiliza o banco NoSQL do tipo grafos, ainda em relação ao motor de recomendação, o recurso “Pessoas que você talvez conheça” da plataforma seria uma demonstração de como funciona o algoritmo de recomendação do tipo grafos. Que se baseia em conexões do usuário para retornar outras conexões da mesma faculdade e trabalho.

### **1.2.1 Cenário nº 2:**

A definição da entrega de um produto em 24h depende da disponibilidade de estoque do centro de distribuição mais próximo do endereço de entrega. Se o cliente optar por essa entrega fast, é necessário realizar a reserva no centro de distribuição e, automaticamente, atualizar o estoque para atender a outros clientes. Nos testes preliminares com o uso do modelo relacional, o desempenho foi frustrante, influenciado principalmente pelo volume de dados e frequência de atualização.

(Para esse segundo cenário, a TI propôs o uso de um banco de dados NoSQL do tipo COLUNAR)

### **1.2.2 Justificativa do cenário:**

O modelo chave-valor é o mais ideal para essa situação pois sua estrutura simplificada permite uma escalabilidade horizontal linear, distribuindo as chaves entre múltiplos servidores (Sharding) sem a complexidade de manter integridade referencial entre tabelas. Isso garante que, mesmo com um aumento repentino no volume de acessos e transações de estoque, o sistema mantenha um tempo de resposta constante e baixíssima latência, evitando os gargalos de processamento e as travas de concorrência típicas dos bancos relacionais em cenários de alta frequência de atualização.

### **1.2.3 Empresa que utiliza o cenário:**

Como por exemplo a Amazon que utiliza o banco do tipo chave-valor, são um dos pioneiros do modelo chave-valor, com o Amazon Dynamo que mais tarde se transformou no DynamoDB. Com o banco relacional a Amazon lidava com o problema de volume de acesso simultâneo em eventos de promoção como a Black Friday, a solução era um banco que lidasse com este volume, assim utilizaram o chave-valor, que cada cliente possui um ID (Chave) que se referencia nos produtos (Valor), assim milhões de pessoas podem adicionar vários produtos ao carrinho e se assegurar que o estoque se atualize ao solicitar um pedido, mesmo que as pessoas realizem a mesma ação ao mesmo tempo também.

### **1.3.1 Cenário nº 3:**

A tela de detalhes de um produto sempre recebe novas informações e, hoje em dia, possui informações que podem ser armazenadas juntamente com o produto,

tais como: reviews do produto; suas versões; informações de entrega; imagens; recomendações; dicas, entre outras.

(Para esse terceiro cenário, a TI propôs o uso de um banco de dados NoSQL do tipo DOCUMENTO)

### **1.3.2 Justificativa do cenário:**

O modelo de banco de dados orientado a documentos (como o MongoDB) é o mais ideal para essa situação devido ao seu esquema flexível (Schemaless), que permite agrupar todas as informações variadas de um produto como reviews, imagens e versões em um único documento JSON/ BSON. Essa abordagem elimina a necessidade de múltiplos “JOINS” complexos entre tabelas separadas, garantindo que todos os dados necessários para a tela de detalhes sejam recuperados em uma única operação de leitura, o que otimiza drasticamente a performance e facilita a evolução contínua do catálogo sem a necessidade de migrações rígidas de estrutura.

### **1.3.3 Empresa que utiliza o cenário:**

Como por exemplo a eBay que utiliza o banco orientado a documentos, para um anúncio são necessárias algumas informações como as fotos, descrição e avaliações, são muitos dados que há em cada anúncio, mas com o banco orientado a documentos todas essas informações são armazenadas em um único documento JSON/ BSON, com a compactação de dados que é realizada o sistema se torna mais leve. Ao haver um evento de sobrecarregamento, o banco orientado a documentos não estoura por razão da distribuição de servidores nativo (Sharding).

## 2.0 ANÁLISE DOS DADOS DE VENDAS

### 2.1 Quantidade

No arquivo existem sim outliers nos dados disponibilizados, é identificado que há uma discrepância em vendas com relação aos valores outliers e os valores gerais. As 11283 linhas que representam os outliers realizaram um pouco mais da metade do valor gerado pelas outras 189503 linhas que não são outliers.

Total de vendas (Normal): 79.775.077,91

Total de vendas (Outliers): 44.564.916,42

```

181 def clean_col(col):
182     return pd.to_numeric(df[col].astype(str).str.replace(',', '.'), errors='coerce')

df['quantidade'] = clean_col('quantidade')
df['valor_total_bruto'] = clean_col('valor_total_bruto')

# Remover nulos da coluna de análise
df_q = df.dropna(subset=['quantidade'])

# 3. Identificação de Outliers (Método IQR)
Q1 = df_q['quantidade'].quantile(0.25)
Q3 = df_q['quantidade'].quantile(0.75)
IQR = Q3 - Q1

limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

outliers = df_q[(df_q['quantidade'] < limite_inferior) | (df_q['quantidade'] > limite_superior)]
dados_normais = df_q[(df_q['quantidade'] >= limite_inferior) & (df_q['quantidade'] <= limite_superior)]

# 4. Cálculos e Justificativa
print(f"Limite Superior para Outliers: {limite_superior}")
print(f"Quantidade de Outliers encontrados: {len(outliers)}")

media_venda_outliers = outliers['valor_total_bruto'].mean()
media_venda_geral = df['valor_total_bruto'].mean()

print(f"Média de venda (Outliers de Qtd): R$ {media_venda_outliers:.2f}")
print(f"Média de venda (Geral): R$ {media_venda_geral:.2f}")

Limite Superior para Outliers: 3.5
Quantidade de Outliers encontrados: 11283
Média de venda (Outliers de Qtd): R$ 3949.74
Média de venda (Geral): R$ 428.97

181 #Estimativa de variabilidade
182 desvio_padrao_filtrado = dados_normais['quantidade'].std()
183 print(f"\nEstimativa de variabilidade (sem outliers): {desvio_padrao_filtrado:.2f}")

Estimativa de variabilidade (sem outliers): 0.78

```

#### 2.1.2 Preço

Não há uma diferença estaticamente significativa entre as médias. Ao realizar a comparação com uma amostra, o resultado está abaixo de 5%, o que não é um valor significativo.

```

140 # 3. Cálculo da média da população
141 media_populacao = df_completo['valor'].mean()
142 print(f"Média da População: {media_populacao:.2f}\n")

143 # 4. Função para realizar o teste t
144 def analisar_significancia(df, coluna_agrupadora, media_alvo):
145     categorias = df[coluna_agrupadora].unique()
146     resultados = []

147     for cat in categorias:
148         dados_cat = df[df[coluna_agrupadora] == cat][['valor']]
149         # Realiza o teste t de uma amostra
150         t_stat, p_val = stats.ttest_1samp(dados_cat, media_alvo)

151         resultados.append({
152             'Categoria': cat,
153             'Média': dados_cat.mean(),
154             'P-valor': p_val,
155             'Significativo (p < 0.05)': 'Sim' if p_val < 0.05 else 'Não'
156         })

157     return pd.DataFrame(resultados)

158 Média da População: 111.09

159

160 import scipy.stats as stats

161 # 5. Execução dos testes
162 res_regiao = analisar_significancia(df_completo, 'regiao_pais', media_populacao)
163 res_pagamento = analisar_significancia(df_completo, 'formapagto', media_populacao)

164 print("Comparação por Região:")
165 print(res_regiao)
166 print("\nComparação por Modalidade de Pagamento:")
167 print(res_pagamento)

168 Comparação por Região:
169   Categoria  Média  P-valor Significativo (p < 0.05)
170 0 Norte     111.184469  0.980435 Não
171 1 Centro Oeste  111.232447  0.984852 Não
172 2 Nordeste     111.096483  0.993941 Não
173 3 Sudeste     111.076545  0.998723 Não
174 4 Sul         110.867752  0.873138 Não

175 Comparação por Modalidade de Pagamento:
176   Categoria  Média  P-valor Significativo (p < 0.05)
177 0 Dinheiro   111.478004  0.719148 Não
178 1 Pix        111.230406  0.895239 Não
179 2 Boleto Bancário  110.585834  0.636612 Não
180 3 Cartão Crédito  112.508030  0.191887 Não
181 4 Cartão Débito   109.638472  0.170567 Não

```

## 2.1.3 Correlação

Valores com forte correlação positiva: “valor\_comissao” e “lucro\_liquido”

Valores com forte correlação negativa: “quantidade” e “valor”

```

182 # Limpeza e conversão de dados numéricos (substituindo ',' por '.')
183 cols_to_convert = ['valor', 'quantidade', 'valor_total_bruto', 'valor_comissao', 'lucro_liquido']
184 for col in cols_to_convert:
185     df_all[col] = pd.to_numeric(df_all[col].astype(str).str.replace(',', '.'), errors='coerce')

186 # Cálculo da matriz e geração do heatmap
187 corr_matrix = df_all[cols_to_convert].dropna().corr()
188 plt.figure(figsize=(10, 8))
189 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
190 plt.title("Matriz de Correlação - Dados Petshop (2019-2022)")
191 plt.savefig('matriz_correlacao.png')

```

